Anglicismos, francesismos y cálculo probabilístico

Una historia conceptual para probabilidad, matemáticas aplicadas y computación

Probabilidad · pseudocódigo · algoritmos · C · lógica formal

César Galindo · 31 de mayo de 2026

Tesis de la presentación

Las palabras técnicas no son adornos: son pequeñas máquinas de pensamiento.

Un anglicismo o un francesismo no entra a la ciencia únicamente por moda. Entra cuando permite nombrar con rapidez una operación: modelar, ajustar, compilar, iterar, simular, optimizar, debuggear.

La historia de estos préstamos revela tres capas:

- una capa institucional: universidades, laboratorios, empresas, revistas;
- una capa técnica: algoritmos, lenguajes, bibliotecas, estándares;
- una capa lógica: reglas de inferencia, invariantes, tipos, precondiciones y pruebas.

Por qué importa en matemáticas aplicadas y computación

En matemáticas aplicadas y computación, el vocabulario no sólo describe objetos; también codifica prácticas de trabajo.

Inglés operativo

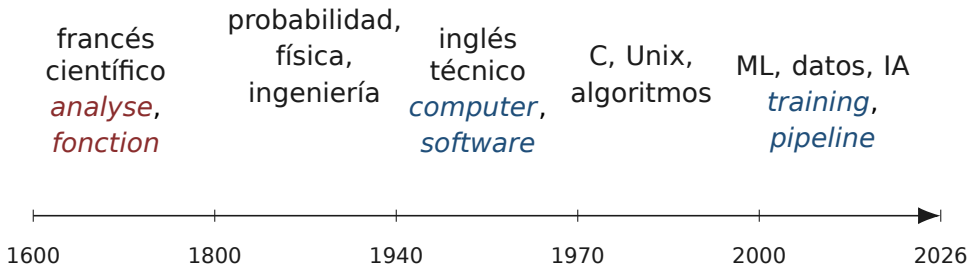
- *fit*: estimar parámetros desde datos;
- *loss*: convertir error en objetivo;
- *pipeline*: encadenar transformaciones;
- *debug*: volver visible una falla lógica.

Francés estructural

- *ensemble*: pensar colecciones como objetos;
- *ordinateur*: máquina que ordena información;
- *logiciel*: programa como objeto lógico;
- *loi*: distribución como medida.

La lengua técnica es una interfaz entre teoría, código y comunidad.

Mapa histórico: del cálculo al código



No hay una sustitución simple: el francés no desaparece y el inglés no lo reemplaza todo. Más bien, cada lengua deja capas: análisis, optimización, informática, software, machine learning.

De la lengua culta a la lengua operativa

Durante siglos, el latín y el francés funcionaron como lenguas de prestigio académico. A partir del siglo XX, el inglés se vuelve la lengua de circulación rápida de artículos, manuales, estándares y documentación.

La diferencia crucial es ésta:

- el francés transmitió muchas categorías de análisis científico moderno: *analyse*, *fonction*, *variable*, *ensemble*, *ordinateur*;
- el inglés técnico transmitió formas de trabajo computacional: *file*, *loop*, *buffer*, *stack*, *debug*, *training*, *benchmark*;
- el español técnico recibe, adapta, traduce o convive con ambos registros.

Francesismos científicos: análisis, función, variable

Muchos términos que hoy parecen universales entraron a la tradición matemática moderna por el francés académico.

Francés	Español técnico	Idea matemática
analyse	análisis	descomponer para comprender
fonction	función	dependencia regulada
variable	variable	cantidad que puede cambiar
limite	límite	frontera de aproximación
dérivée	derivada	tasa local de variación
équation	ecuación	igualdad con estructura

El francesismo no siempre se nota porque fue naturalizado: muchas palabras ya parecen “matemáticas puras”.

La palabra *ensemble*: de Cantor a Bourbaki

Ensemble significa conjunto. En el siglo XX, la escuela francesa ayudó a consolidar una visión estructural de la matemática: conjuntos, aplicaciones, relaciones, estructuras algebraicas, espacios topológicos.

En computación, esta herencia aparece en:

- tipos de datos abstractos: conjuntos, listas, árboles, grafos;
- bases de datos: relaciones y operaciones sobre colecciones;
- aprendizaje automático: *ensemble methods*, árboles combinados, *bagging*, *boosting*;
- lógica: modelos como estructuras con universos y relaciones.

La palabra viaja: de *ensemble* matemático a *ensemble learning* computacional.

Francia y la informática: *ordinateur*, *logiciel*, *informatique*

La tradición francesa produjo términos alternativos al inglés técnico.

- *ordinateur*: propuesto en Francia en 1955 para nombrar al *computer*; la idea no es sólo calcular, sino ordenar información.
- *logiciel*: derivado de *logique*; se propone frente a *software* para designar un conjunto estructurado de programas.
- *informatique*: combinación de información y automática; designa el tratamiento automático de la información.

Referencias: Académie française, *logiciel*; carta de Jacques Perret sobre *ordinateur*; historia de *informatique* asociada a Philippe Dreyfus.

Anglicismos computacionales: no son sólo palabras

Los anglicismos de computación suelen venir pegados a una práctica concreta.

Anglicismo	Traducción posible	Práctica técnica
debug	depurar	encontrar fallas lógicas
compile	compilar	traducir lenguaje fuente
runtime	tiempo de ejecución	comportamiento dinámico
stack	pila	memoria/estructura LIFO
heap	montículo/heap	memoria dinámica
hash	dispersión/hash	acceso por clave

A veces traducimos; a veces adoptamos; a veces vivimos con las dos formas.

Anglicismo como economía cognitiva

Decir *pipeline* en una clase de datos no es solamente decir “tubería”. Es invocar una arquitectura:

$$X_{\text{raw}} \longrightarrow X_{\text{clean}} \longrightarrow Z_{\text{features}} \longrightarrow \widehat{y} \longrightarrow \text{report}.$$

El anglicismo comprime una secuencia completa:

- ingestión de datos;
- limpieza y transformación;
- entrenamiento o ajuste;
- validación;
- despliegue o reporte.

Una buena traducción debe conservar la operación, no sólo cambiar la palabra.

Pseudocódigo: una lengua puente

El pseudocódigo es una lengua híbrida: no es español, no es inglés, no es C, no es matemática pura. Es una gramática intermedia para expresar procesos.

- Hereda de la matemática: variables, condiciones, cuantificadores, funciones.
- Hereda del inglés técnico: *if*, *while*, *return*, *input*, *output*.
- Hereda de la lógica: implicaciones, invariantes, casos, terminación.
- Hereda de la ingeniería: claridad, modularidad, prueba y costo computacional.

Por eso el pseudocódigo es una forma de pensamiento algorítmico antes de ser programación.

Ejemplo I: algoritmo de Euclides en pseudocódigo

```
Algorithm Euclid(a,b)
Input: integers a >= b > 0
Output: gcd(a,b)
while b != 0 do
    r <- a mod b
    a <- b
    b <- r
return a
```

La forma *while*, *return*, *input*, *output* no es accidental: reproduce la convención internacional de los manuales de algoritmos. La matemática, en cambio, vive en el invariante:

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Lógica detrás del algoritmo de Euclides

La corrección se divide en dos partes.

Invariante. En cada iteración:

$$\gcd(a_{\text{old}}, b_{\text{old}}) = \gcd(a_{\text{new}}, b_{\text{new}}).$$

Terminación. El segundo argumento disminuye:

$$0 \leq r < b.$$

Por buen orden de los naturales, el proceso termina.

La palabra *loop* nombra una repetición; la lógica dice por qué esa repetición no es ciega.

Ejemplo II: Euclides en C

```c
#include <stdio.h>

int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

C no sólo implementa el pseudocódigo: lo obliga a tomar decisiones de tipo, memoria, operadores y flujo de control. El algoritmo se vuelve máquina.

C: una lengua de sistema con vocabulario inglés mínimo

C nace en Bell Labs en el entorno de Unix. Dennis Ritchie transformó B en C durante 1971–1973, añadiendo tipos y otros cambios importantes; C se volvió una lengua crucial para escribir Unix y software portable.

- *int*, *char*, *return*, *while*, *switch*: palabras cortas, operativas.
- Sintaxis austera: pocas palabras reservadas, mucha responsabilidad del programador.
- Cercanía con memoria: apuntadores, direcciones, arreglos, representación.

Referencia histórica: Dennis M. Ritchie, “The Development of the C Language”, Bell Labs, 1993.

Del inglés de C al español técnico

En la práctica docente se mezclan registros:

C / inglés	Español de aula	Concepto lógico
if	si	selección por verdad
while	mientras	repetición condicionada
return	devolver	salida de una función
break	romper/salir	corte del flujo
pointer	apuntador	referencia a memoria
array	arreglo	secuencia indexada

No hay que prohibir el anglicismo: hay que hacerlo consciente y matemáticamente preciso.

Lógica proposicional dentro de C

```c
if ((x >= 0) && (x <= 1)) {
    y = sqrt(x);
} else {
    printf("input outside domain\n");
}
```

La condición es una fórmula:

$$(0 \leq x) \wedge (x \leq 1).$$

El *if* de C es una puerta lógica: ejecuta una rama si una proposición es verdadera. La programación elemental ya contiene semántica formal.

Precondiciones y postcondiciones

Una función computacional puede leerse como una proposición de Hoare:

$$\{P\}\ C\ \{Q\}.$$

- P: precondición, lo que debe cumplirse antes.
- C: comando o programa.
- Q: postcondición, lo que queda garantizado después.

Ejemplo:

$$\{a, b > 0\}\ \texttt{gcd(a,b)}\ \{\texttt{return} = \gcd(a, b)\}.$$

El algoritmo deja de ser receta y se vuelve teorema ejecutable.

Pseudocódigo con invariante explícito

```
Algorithm BinarySearch(A,x)
Input: sorted array A[0..n-1], value x
Output: index i such that A[i]=x, or -1
lo <- 0; hi <- n-1
while lo <= hi do
    invariant: if x occurs, then it occurs in A[lo..hi]
    mid <- floor((lo + hi)/2)
    if A[mid] == x then return mid
    if A[mid] < x then lo <- mid + 1
    else hi <- mid - 1
return -1
```

Search, búsqueda y complejidad

Binary search se traduce como búsqueda binaria. La palabra inglesa *search* enfatiza la acción; el análisis matemático enfatiza el costo.

$$T(n) = T(n/2) + O(1), \qquad T(n) = O(\log n).$$

Aquí convergen tres lenguas:

- inglés técnico: *binary search*;
- español matemático: recurrencia y complejidad logarítmica;
- lógica: el invariante asegura que no se pierde la solución.

FORTRAN, ALGOL, C: tres estilos de lengua técnica

Lenguaje	Época	Nombre	Estilo
FORTRAN	1950s	*Formula Translation*	cálculo científico
ALGOL	1958/60	*Algorithmic Language*	estructura algorítmica
C	1970s	letra heredada de B	sistemas y portabilidad

ALGOL distinguió entre lenguaje de referencia, publicación y representaciones de hardware. Esa separación anticipa una idea central: no todo código es sólo máquina; también es notación, comunicación y estándar.

Referencia: Report on the Algorithmic Language ALGOL 60, CACM, 1960.

Anglicismos en estadística: *fit*, *sample*, *bias*

La estadística moderna en español vive con un doble registro:

Inglés	Español	Fórmula o idea
sample	muestra	datos observados
fit	ajuste	$\widehat{\theta} = \arg\min L(\theta)$
bias	sesgo	$\mathbb{E}\widehat{\theta} - \theta$
variance	varianza	dispersión muestral
bootstrap	remuestreo	aproximar incertidumbre
baseline	línea base	comparación mínima

El préstamo se justifica cuando ayuda a entrar en una comunidad de métodos.

Loss: una palabra inglesa para una idea variacional

En aprendizaje automático se minimiza una *loss function*. En español decimos pérdida, función de pérdida o función de costo.

$$\widehat{\theta} \in \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f_\theta(x_i)).$$

La palabra *loss* concentra una genealogía:

- error de predicción;
- riesgo estadístico;
- optimización numérica;
- entrenamiento iterativo.

No es sólo un anglicismo: es una filosofía computacional de estimación.

Pseudocódigo: descenso por gradiente

```
Algorithm GradientDescent(theta0, eta, N)
Input: initial parameter theta0, learning rate eta
Output: approximate minimizer theta
theta <- theta0
for k = 0,...,N-1 do
    g <- gradient L(theta)
    theta <- theta - eta*g
return theta
```

Learning rate convive con tasa de aprendizaje. El término inglés es dominante porque viene del ecosistema de ML, pero la idea matemática es clásica: paso, gradiente y descenso.

Training: entrenamiento como inferencia computacional

En ML, *training* no significa sólo “practicar”. Significa resolver una tarea de inferencia mediante optimización.

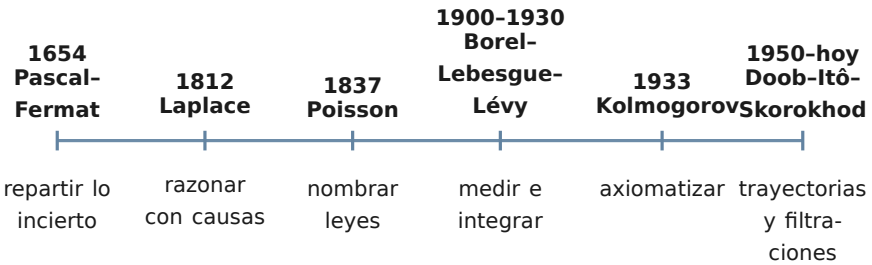
$$\text{datos} \longrightarrow \text{parámetros} \longrightarrow \text{predicción}.$$

La palabra organiza un ritual técnico:

- dividir datos en *train*, *validation*, *test*;
- ajustar hiperparámetros;
- medir *generalization*;
- evitar *overfitting*;
- reportar *benchmarks*.

Probabilidad y francés: una genealogía central

La probabilidad moderna no nació como una lista de fórmulas. Nació como una transformación lingüística y conceptual: del *hasard* de los juegos al cálculo de leyes, esperanzas y trayectorias.



La línea profunda es ésta: juego → ley → medida → proceso → algoritmo. El francés aporta vocablos decisivos; la escuela soviética aporta el gesto axiomático y la notación de una teoría general.

No es glosario: es arquitectura conceptual

La tradición francesa introduce palabras que funcionan como operadores matemáticos.

Escena de azar

- *hasard*: lo contingente, lo no controlado;
- *chance*: posibilidad favorable;
- *parti*: reparto justo de una partida interrumpida;
- *espérance*: ganancia anticipada.

Objeto matemático

- evento $A \in \mathcal{F}$;
- probabilidad $\mathbb{P}(A)$;
- ley $\mathbb{P}_X$;
- esperanza $\mathbb{E}[X]$.

La palabra francesa conserva el rastro histórico, pero la notación moderna la disciplina: evento, medida, integral, condicionalidad.

Pascal–Fermat: recursión antes de la computadora

El problema de los puntos pregunta cómo repartir una apuesta cuando el juego se interrumpe. La idea profunda es mirar los futuros posibles, no sólo el pasado observado.

$$V(a,b) = \frac{1}{2}V(a+1,b) + \frac{1}{2}V(a,b+1),$$

con condiciones de frontera cuando uno de los jugadores ya ganó.

Aquí aparecen tres ideas que luego serán computacionales:

- estado: marcador parcial de la partida;
- recursión: valor actual como promedio de valores futuros;
- programa dinámico: resolver desde la frontera hacia atrás.

La correspondencia Pascal–Fermat de 1654 fija una forma temprana de cálculo de expectativas condicionadas.

Bernoulli: el libro que cambia la escala del azar



JACOBI BERNOULLI,
Profess. Basil. & utriusque Societ. Reg. Scientiar.
Gall. & Prussˢ. Sodal.
Mathematici Celeberrimi,

ARS CONJECTANDI,

OPUS POSTHUMUM.

Accedit

TRACTATUS
DE SERIEBUS INFINITIS,

Et Epistola Gallicè scripta
DE LUDO PILÆ
RETICULARIS.

BASILEÆ,
Impensis THURNISIORUM, Fratrum.
cIↃ IↃcc xiii.

Título. *Ars conjectandi, opus posthumum*. Basilea, 1713.

Autor. Jacob Bernoulli; publicado después de su muerte.

Lengua principal. Latín académico: la lengua internacional del tratado erudito.

Detalle revelador. La portada anuncia también una *epistola Gallice scripta*: una carta escrita en francés.

La portada muestra la transición: el núcleo doctrinal todavía habla latín; el francés aparece como lengua viva de intercambio científico.

Imagen: Wikimedia Commons, reproducción de dominio público de la portada de 1713.

Ars conjectandi: por qué no es sólo “cálculo de juegos”

El título no dice simplemente “probabilidad”. Dice *arte de conjeturar*. En el siglo XVII, *conjeturar* no significa adivinar sin método: significa razonar cuando la información es incompleta.

La estructura conceptual del libro es decisiva:

- parte de juegos de azar, combinatoria y esperanza;
- organiza técnicas dispersas en una doctrina de cálculo;
- separa azar observado, regularidad esperada y prueba matemática;
- culmina en el teorema áureo: una forma temprana de la ley de los grandes números.

Por eso Bernoulli es una bisagra: del problema local de apuestas pasa a la idea de que las frecuencias empíricas pueden estabilizarse con alta probabilidad.

La importancia: de la frecuencia al límite probabilístico

La intuición central puede escribirse hoy así. Si

$$X_1, \dots, X_n \sim \text{Bernoulli}(p) \quad \text{independientes}, \qquad \bar{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i,$$

entonces, para todo $\varepsilon > 0$,

$$\mathbb{P}(|\bar{X}_n - p| < \varepsilon) \longrightarrow 1.$$

Esta fórmula moderna traduce una idea histórica formidable: la incertidumbre individual no desaparece, pero el promedio de muchas repeticiones puede obedecer una regularidad cuantificable.

Ahí nace una gramática que todavía usamos en estadística, simulación Monte Carlo y aprendizaje automático: muestra, promedio empírico, error, concentración y confianza.

Lenguas superpuestas: latín, francés y probabilidad

Bernoulli permite ver que la historia lingüística de la probabilidad no es lineal.

- El latín da forma de tratado: autoridad, sistematicidad, circulación erudita.
- El francés aparece como lengua de correspondencia, conversación científica y, después, de análisis probabilístico.
- El alemán será crucial para la axiomatización de Kolmogorov en 1933.
- El inglés dominará la documentación computacional: *random variable*, *sampling*, *Monte Carlo*, *training*.

La lección para el Beamer es importante: cada lengua no sólo traduce términos; transporta instituciones, géneros de escritura y estilos de razonamiento.

Pseudocódigo: problema de los puntos

```
Algorithm PointsValue(a,b,r)
Input: current scores a,b; target r
Output: fair share for player A
if a == r then return 1
if b == r then return 0
return 0.5*PointsValue(a+1,b,r)
     + 0.5*PointsValue(a,b+1,r)
```

Este pseudocódigo traduce una intuición francesa clásica —repartir justamente la *partie*— en una estructura algorítmica: frontera, recursión, esperanza condicional.

Espérance: de ganancia anticipada a integral

La palabra *espérance* parece cotidiana, pero en probabilidad se vuelve una operación de integración.

$$\mathbb{E}[X] = \sum_x x\,\mathbb{P}(X = x), \qquad \mathbb{E}[X] = \int_{\mathbb{R}} x\,\mathbb{P}_X(dx).$$

Obsérvese la evolución conceptual:

- en juegos: cantidad que conviene esperar antes de jugar;
- en estadística: promedio teórico detrás de los datos;
- en ML: riesgo esperado $\mathbb{E}[\ell(f(X), Y)]$;
- en simulación: límite de promedios Monte Carlo.

En francés moderno: *espérance mathématique*; en inglés: *expected value*; en notación soviética frecuente: $M\xi$.

Laplace: probabilidad como razón aplicada

Con Laplace, la probabilidad se vuelve una lengua general para razonar bajo información incompleta: causas, errores de observación, astronomía, demografía, decisiones.

$$\mathbb{P}(H \mid D) = \frac{\mathbb{P}(D \mid H)\mathbb{P}(H)}{\mathbb{P}(D)}.$$

La capa lingüística es reveladora:

- *probabilité a priori*: estructura previa del modelo;
- *probabilité a posteriori*: actualización por datos;
- *likelihood*: función de verosimilitud en el lenguaje estadístico moderno;
- *Bayesian update*: versión computacional del razonamiento laplaciano.

Poisson, Borel, Lebesgue, Lévy: de la ley a la medida

La palabra francesa *loi* es central: una variable aleatoria no sólo toma valores; tiene una ley.

$$X \sim \mu \quad \Longleftrightarrow \quad \mathbb{P}(X \in A) = \mu(A), \qquad A \in \mathcal{B}(\mathbb{R}).$$

La genealogía técnica avanza así:

- Poisson: leyes para conteos raros y aproximaciones;
- Borel: eventos como conjuntos medibles;
- Lebesgue: integral como maquinaria de esperanza;
- Lévy: procesos, saltos, estabilidad e infinitamente divisibles;
- Bachelier: fluctuación aleatoria como modelo financiero.

Loi: una palabra francesa para distribución

En francés se dice *loi de probabilité*. La expresión es más fuerte que "distribución": sugiere una regularidad interna del azar.

ley discreta: $\mathbb{P}_X = \sum_i p_i \delta_{x_i}$,

ley continua: $\mathbb{P}_X(dx) = f(x)\,dx$.

- *loi normale*: errores agregados;
- *loi de Poisson*: eventos raros;
- *loi exponentielle*: espera sin memoria;
- *loi empirique*: datos como medida.

Una palabra francesa prepara la notación moderna: no escribir sólo densidades, sino medidas $\mathbb{P}_X(dx)$.

Función de repartición: por qué no basta decir distribución

En la tradición francesa, la *fonction de répartition* nombra una función muy precisa:

$$F_X(x) = \mathbb{P}(X \le x), \qquad x \in \mathbb{R}.$$

Decir función de distribución no es fatal, porque es uso común en español. Pero es menos limpio por tres razones.

- Distribución puede significar la ley completa $\mathbb{P}_X$, no sólo su acumulada.
- En análisis, distribución también puede significar distribución generalizada en el sentido de Schwartz.
- La palabra repartición conserva mejor la idea acumulativa: cuánto de la masa probabilística queda repartida hasta x.

Por eso, para evitar ambigüedad, es elegante escribir: función de repartición para F_X, y ley o distribución de probabilidad para $\mathbb{P}_X$.

Ley, repartición y cuantiles: tres objetos distintos

La claridad terminológica evita errores en teoría y en código.

$$\begin{aligned} \text{ley:} \quad & \mu_X = \mathbb{P}_X, \\ \text{repartición:} \quad & F_X(x) = \mu_X((-\infty, x]), \\ \text{cuantil:} \quad & F_X^{-1}(u) = \inf\{x : F_X(x) \geq u\}. \end{aligned}$$

Estos objetos contienen información relacionada, pero no cumplen el mismo papel.

- La *loi* vive en conjuntos medibles: $\mu_X(A)$.
- La función de repartición vive en la recta ordenada: $F_X(x)$.
- El cuantil vive en simulación, riesgo, VaR, bootstrap e inversión numérica.

En programación, confundirlos produce APIs imprecisas: `cdf(x)` no es `law(A)`, y `quantile(u)` no es una densidad.

Masa de probabilidad no es densidad de Lebesgue

Llamar función de densidad a una función de masa de probabilidad es, en cursos básicos, una mala traducción.

Si X es discreta,

$$p_X(k) = \mathbb{P}(X = k), \qquad \sum_k p_X(k) = 1.$$

Esto es una función de masa de probabilidad. Tiene átomos: la probabilidad de un punto puede ser positiva.

Si X es continua respecto a Lebesgue,

$$\mathbb{P}(X \in A) = \int_A f_X(x)\,dx, \qquad \mathbb{P}(X = x) = 0.$$

Esto sí es una densidad respecto a dx.

La diferencia no es de estilo: cambia sumas por integrales, puntos por intervalos, y masa puntual por densidad local.

La versión correcta: densidad respecto a una medida

La teoría de la medida permite decirlo con precisión quirúrgica:

$$\mathbb{P}_X(dx) = g(x)\,\nu(dx), \qquad g = \frac{d\mathbb{P}_X}{d\nu}.$$

Entonces una masa discreta $p_X(k)$ puede verse como densidad sólo si se aclara la medida de referencia:

$$\mathbb{P}_X = \sum_k p_X(k)\delta_k, \qquad \mathbb{P}_X(dk) = p_X(k)\,\#(dk),$$

$\#$ siendo la medida de conteo.

Pero en probabilidad elemental, decir simplemente “densidad” sugiere densidad respecto a Lebesgue. Por eso la nomenclatura recomendable es:

discreta: masa continua: densidad general: ley/medida.

Pseudocódigo: no mezclar PMF, CDF y densidad

```
Algorithm IdentifyProbabilityObject(object Q)
Input: symbolic or numerical probability object Q
Output: semantic label
if Q returns P(X = x) for isolated x then
    return "mass function / PMF"
if Q returns P(X <= x) then
    return "repartition function / CDF"
if Q integrates over intervals with respect to dx then
    return "Lebesgue density"
if Q assigns probabilities to measurable sets A then
    return "law / probability measure"
```

La lógica pedagógica es simple: primero identificar la semántica del objeto; después elegir la palabra. La traducción no debe borrar la estructura matemática.

La escuela soviética: el giro axiomático

El punto de inflexión llega con Kolmogorov: la probabilidad deja de depender de intuiciones de juego, frecuencia o simetría y se instala dentro de la teoría de la medida.

$$(\Omega, \mathcal{F}, \mathbb{P}), \qquad X : (\Omega, \mathcal{F}) \longrightarrow (E, \mathcal{E}).$$

La escuela soviética no borra la tradición francesa: la reorganiza.

- *loi* se convierte en medida imagen: $\mathbb{P}_X = \mathbb{P} \circ X^{-1}$;
- *espérance* se convierte en integral: $\mathbb{E}[X] = \int X(\omega)\, \mathbb{P}(d\omega)$;
- eventos, independencia y condicionalidad se formulan axiomáticamente.

Kolmogorov, *Grundbegriffe der Wahrscheinlichkeitsrechnung* (1933).

Genealogía franco–soviética de la notación

No conviene poner aquí una tabla: la historia no son dos columnas, sino una migración de conceptos.

Primero, la tradición francesa fija palabras de modelación:

loi, *espérance*, *martingale*, *fonction de répartition*.

Después, la tradición rusa y soviética exige una gramática axiomática:

$$(\Omega, \mathcal{F}, P), \qquad \xi : \Omega \to \mathbb{R}, \qquad F_\xi(x) = P\{\omega : \xi(\omega) \le x\}.$$

El cambio de estilo es profundo: ya no se pregunta sólo “cuál es la fórmula”, sino sobre qué espacio vive, con qué sigma-álgebra se mide, qué información se conoce, y bajo qué límite converge.

Estilo soviético: variables, operadores y eventos

En muchos textos rusos y soviéticos, la notación enfatiza que la variable aleatoria es una función medible, no una simple letra numérica.

$$\xi = \xi(\omega), \qquad M\xi = \int_\Omega \xi(\omega)\, P(d\omega), \qquad D\xi = M(\xi - M\xi)^2.$$

También es frecuente escribir eventos con llaves:

$$\{\xi \le x\}, \qquad P\{\xi \le x\}, \qquad F_\xi(x) = P\{\xi \le x\}.$$

La notación obliga a recordar que detrás de cada desigualdad hay un subconjunto de Ω. Esa pedagogía es extraordinariamente útil para evitar errores con condicionalidad, independencia y convergencia.

De Francia a Moscú: de la ley al espacio de probabilidad

La palabra francesa *loi* no desaparece. Se vuelve una medida imagen:

$$\mu_\xi(B) = P(\xi \in B) = P(\xi^{-1}(B)), \qquad B \in \mathcal{B}(\mathbb{R}).$$

Esta fórmula es una bisagra histórica:

- conserva el lenguaje francés de la ley;
- lo traduce a teoría de la medida;
- permite variables en espacios generales;
- prepara procesos estocásticos, medidas empíricas y aprendizaje estadístico.

En computación, esto se vuelve natural: un objeto `Distribution` no es sólo una función; debe saber muestrear, evaluar probabilidades, acumular, estimar cuantiles y componer transformaciones.

La escuela polaca y el francés: una estrategia internacional

La escuela polaca no escribía en francés por ornamentación. Lo hacía porque el francés era una de las grandes lenguas internacionales de la matemática europea, junto con el alemán y, crecientemente, el inglés.

En el programa polaco de entreguerras había una decisión institucional muy consciente: publicar en lenguas internacionales para que Varsovia y Lwów no fueran periferias, sino centros reconocibles de investigación.

Esto explica la política multilingüe de revistas como *Fundamenta Mathematicae* y *Studia Mathematica*: el objetivo era atraer lectores, árbitros y autores de toda Europa.

Escribir en francés era una forma de circulación científica, no una renuncia a la identidad matemática polaca.

Polonia, Francia y notación: de los conjuntos a la probabilidad

El vínculo con el francés fue natural porque varias áreas cultivadas por la escuela polaca dialogaban con tradiciones francesas:

$$\text{conjuntos} \longrightarrow \text{topología} \longrightarrow \text{medida} \longrightarrow \text{probabilidad}.$$

Sierpiński, Kuratowski, Banach, Steinhaus y sus círculos trabajaban en un ecosistema donde la precisión de la notación importaba tanto como el resultado.

Esa cultura dialoga naturalmente con la probabilidad moderna:

$$A \in \mathcal{F}, \qquad P(A), \qquad X^{-1}(B), \qquad \mu_X(B) = P(X \in B).$$

La lección: una comunidad matemática elige lengua cuando elige audiencia, tradición y velocidad de circulación.

Chebyshev, Markov, Bernstein, Khinchin, Gnedenko

La escuela rusa y soviética hizo de la probabilidad una teoría de límites, dependencia e independencia aproximada.

- Chebyshev: desigualdades y control de desviaciones;
- Markov: cadenas, dependencia local y transición;
- Bernstein: desigualdades, polinomios, aproximación y probabilidad;
- Khinchin: sumas de variables independientes e infinitamente divisibles;
- Gnedenko: teoremas límite, extremos y dominios de atracción.

La idea profunda es que la probabilidad ya no pregunta sólo “¿cuánto vale el azar?”, sino “¿qué estructura emerge cuando muchos azares interactúan?”.

Notación de filtración: la información como objeto

La escuela moderna de procesos estocásticos hereda este giro: ya no basta decir que algo es aleatorio; hay que especificar qué información está disponible en cada tiempo.

$$\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \cdots \subseteq \mathcal{F}_n \subseteq \cdots, \qquad \mathbb{E}[M_{n+1} \mid \mathcal{F}_n] = M_n.$$

La palabra francesa *martingale* se vuelve un concepto axiomático:

- proceso adaptado a una filtración;
- esperanza condicional bien definida;
- ausencia de ganancia previsible;
- base de finanzas matemáticas, concentración y aprendizaje online.

Pseudocódigo: ley empírica y notación soviética

```
Algorithm EmpiricalLaw(sample x[1..n])
Input: observations x_1,...,x_n
Output: empirical probability measure mu_n
mu_n <- zero measure
for i = 1 to n do
    mu_n <- mu_n + (1/n) * delta_{x[i]}
return mu_n
```

En lenguaje francés: construir la *loi empirique*. En lenguaje soviético/axiomático: aproximar una medida μ por

$$\mathbb{P}_n(dx) = \frac{1}{n} \sum_{i=1}^{n} \delta_{X_i}(dx).$$

C: simular una *loi exponentielle*

```c
#include <stdlib.h>
#include <math.h>

static double uniform01(void) {
    return ((double) rand() + 1.0) / ((double) RAND_MAX + 2.0)
    ;
}

double exponential(double lambda) {
    double u = uniform01();
    return -log(1.0 - u) / lambda;
}
```

La función en C es corta; la teoría detrás es profunda:

$$U \sim \mathrm{Unif}(0,1), \qquad X = -\frac{\log(1-U)}{\lambda} \sim \mathrm{Exp}(\lambda).$$

Pseudocódigo: simular una ley de Bernoulli

```
Algorithm Bernoulli(p)
Input: probability p in [0,1]
Output: random variable X in {0,1}
U <- Uniform(0,1)
if U <= p then
    return 1
else
    return 0
```

La lógica es binaria, pero la semántica es probabilística:

$$\mathbb{P}(X = 1) = p, \qquad \mathbb{P}(X = 0) = 1 - p.$$

El *if* implementa una partición de $[0, 1]$; la medida de cada intervalo produce la probabilidad.

C: una caminata aleatoria elemental

```c
#include <stdlib.h>

int step(double p) {
    double u = (double) rand() / (double) RAND_MAX;
    if (u <= p) return 1;
    else return -1;
}

int random_walk(int N, double p) {
    int x = 0;
    for (int k = 0; k < N; ++k) x += step(p);
    return x;
}
```

Caminata aleatoria: la prueba vive fuera del código

El código anterior produce una trayectoria. La teoría dice qué significa.

$$S_N = \sum_{k=1}^{N} \xi_k, \qquad \mathbb{P}(\xi_k = 1) = p, \qquad \mathbb{P}(\xi_k = -1) = 1 - p.$$

Entonces:

$$\mathbb{E}[S_N] = N(2p - 1), \qquad \mathrm{Var}(S_N) = 4Np(1 - p).$$

El programa habla en inglés mínimo: `return`, `for`, `if`. La interpretación habla en francés probabilístico: *loi*, *espérance*, *variance*, *marche aléatoire*.

Martingale: del juego justo a la filtración

Martingale es una palabra con historia de juegos y estrategias. En probabilidad moderna nombra un proceso cuyo futuro esperado, dado el pasado, no tiene ganancia sistemática.

$$\mathbb{E}[M_{n+1} \mid \mathcal{F}_n] = M_n.$$

La palabra conecta:

- juegos de azar y apuestas;
- información creciente: $\mathcal{F}_n$;
- procesos estocásticos;
- finanzas matemáticas: ausencia de arbitraje;
- algoritmos: estimadores secuenciales y garantías de concentración.

Trayectorias càdlàg: un francesismo que sobrevivió intacto

Càdlàg abrevia *continue à droite, limites à gauche*: continua por la derecha y con límites por la izquierda.

Para un proceso $(X_t)_{t\geq 0}$, una trayectoria càdlàg satisface

$$X_t = \lim_{s \downarrow t} X_s, \qquad X_{t-} = \lim_{s \uparrow t} X_s \text{ existe.}$$

La notación de saltos es entonces:

$$\Delta X_t = X_t - X_{t-}.$$

La palabra francesa se mantuvo incluso en inglés porque nombra con precisión una geometría del azar: caminos que pueden saltar, pero no se rompen de cualquier manera.

Por qué càdlàg importa en probabilidad aplicada

Las trayectorias càdlàg son el lenguaje natural para procesos con saltos.

- Proceso de Poisson: cuenta eventos y salta de uno en uno.
- Procesos de Lévy: combinan deriva, ruido gaussiano y saltos.
- Cadenas de Markov en tiempo continuo: permanecen constantes y saltan.
- Finanzas: precios con discontinuidades, defaults y shocks.
- Simulación: una trayectoria debe tener semántica temporal, no sólo una lista de números.

El espacio $D([0, T], E)$ de funciones càdlàg, con topología de Skorokhod, permite hablar de convergencia de procesos con saltos.

Pseudocódigo: detectar saltos en una trayectoria

```
Algorithm DetectJumps(times t[0..n], values x[0..n], eps)
Input: sampled cadlag path and jump threshold eps
Output: approximate jump times
J <- empty list
for i = 1 to n do
    delta <- x[i] - x[i-1]
    if abs(delta) > eps then
        append t[i] to J
return J
```

El código detecta saltos aproximados. La teoría dice qué se está aproximando:

$$\Delta X_t = X_t - X_{t-}.$$

En una trayectoria càdlàg, el valor en t pertenece al “después” del salto; el límite izquierdo pertenece al “antes”.

Bachelier y Lévy: francés, azar y finanzas modernas

Bachelier escribió en 1900 *Théorie de la spéculation*, una tesis temprana donde el mercado se modela con fluctuaciones aleatorias. Lévy, décadas después, ayudó a organizar la teoría de procesos con incrementos independientes y saltos.

$$X_{t+s} - X_t \sim X_s, \qquad \text{incrementos independientes.}$$

Aquí el francés aporta nombres propios que se volvieron estructuras:

- Bachelier: probabilidad y mercado;
- Lévy: saltos, estabilidad, trayectorias;
- Poisson: conteos aleatorios;
- Borel–Lebesgue: eventos medibles e integración.

Probabilidad, francés e inglés computacional

La práctica contemporánea mezcla dos herencias, pero no como traducciones uno a uno. El francés conserva una memoria conceptual; el inglés organiza la interfaz computacional.

Capa probabilística

- *loi*: medida que gobierna a X;
- *espérance*: integral de una variable;
- *martingale*: juego justo con filtración;
- *marche aléatoire*: suma de incrementos.

Capa computacional

- *distribution*: objeto de biblioteca;
- *expected loss*: función objetivo;
- *online learning*: actualización secuencial;
- *random walk*: algoritmo de exploración.

Un mismo algoritmo puede escribir `random`, estimar una *loi*, converger por Kolmogorov y justificarse por una desigualdad de Chebyshev.

Francesismo probabilístico: Monte Carlo

Monte Carlo no nombra una ciudad cualquiera: nombra una imaginación probabilística. La simulación sustituye una integral difícil por una media aleatoria.

$$I = \int_0^1 f(x)\,dx, \qquad \hat{I}_N = \frac{1}{N}\sum_{i=1}^{N} f(U_i), \quad U_i \sim \mathrm{Unif}(0,1).$$

El término francés entra a la computación como símbolo de azar organizado: usar números pseudoaleatorios para resolver problemas deterministas o de alta dimensión.

C: simulación Monte Carlo elemental

```c
#include <stdlib.h>
#include <math.h>

double uniform01(void) {
    return (double) rand() / (double) RAND_MAX;
}

double estimate_mean(int N) {
    double s = 0.0;
    for (int i = 0; i < N; ++i) {
        double u = uniform01();
        s += exp(-u*u);
    }
    return s / N;
}
```

Lógica probabilística del Monte Carlo

La corrección no es exacta sino asintótica:

$$\hat{I}_N \xrightarrow[N\to\infty]{a.s.} I.$$

El programa produce una estimación; la probabilidad produce una garantía. La palabra *random* en código no basta: hay que saber qué distribución se está simulando y qué teorema justifica el promedio.

Aquí la lengua técnica conecta tres planos:

- simulación computacional;
- ley de los grandes números;
- control de error estadístico.

Benchmark: una institución lingüística

Un *benchmark* no es sólo una prueba. Es una convención social que permite comparar métodos.

- En algoritmos: tiempo, memoria, complejidad empírica.
- En ML: precisión, pérdida, AUC, F1, error fuera de muestra.
- En sistemas: latencia, throughput, consumo energético.
- En estadística: sesgo, varianza, cobertura, robustez.

La palabra inglesa se conserva porque nombra una institución de comparación reproducible.

Overfitting: cuando la palabra ya es teoría

Overfitting suele traducirse como sobreajuste. Pero el anglicismo se impuso porque señala un diagnóstico completo:

bajo error de entrenamiento + alto error de prueba.

En lenguaje estadístico:

- exceso de flexibilidad;
- varianza alta;
- mala generalización;
- ajuste al ruido.

Una palabra concentra una patología de la inferencia.

Backpropagation: cálculo diferencial en inglés computacional

Backpropagation no es una palabra aislada; es un programa de cálculo automático.

$$\frac{\partial L}{\partial w_{ij}^{(\ell)}} = \delta_i^{(\ell)} a_j^{(\ell-1)}.$$

El término une:

- regla de la cadena;
- grafos computacionales;
- memoria de activaciones;
- entrenamiento por gradiente.

En español podemos decir retropropagación, pero en documentación técnica conviven ambos registros.

Pseudocódigo lógico: clasificador binario

```
Algorithm ThresholdClassifier(x, tau)
Input: score x, threshold tau
Output: class label in {0,1}
if x >= tau then
    return 1
else
    return 0
```

Detrás de esta forma mínima hay una fórmula:

$$\widehat{y} = \mathbf{1}_{\{x \geq \tau\}}.$$

La palabra *threshold* traduce una frontera lógica: verdadero/falso, aceptar/rechazar, positivo/negativo.

La lógica de los tipos

C obliga a declarar tipos:

`int`, `double`, `char`, `struct`.

Los tipos son una forma de lógica: restringen qué operaciones tienen sentido.

- Un `int` no es un número real matemático.
- Un `double` es una aproximación finita.
- Un `char*` puede representar texto, pero también una dirección peligrosa.
- Un `struct` construye objetos compuestos.

La lengua de C hace visible la tensión entre abstracción matemática y representación material.

C y memoria: el anglicismo *pointer*

```c
void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Un *pointer* no es sólo una “flecha”. Es una dirección de memoria que permite modificar un objeto indirectamente.

nombre ⟶ dirección ⟶ valor.

Aquí el anglicismo conserva una semántica técnica muy precisa.

Cache, *cache*: préstamo doble

Cache viene del francés “escondite”. En computación, *cache* designa memoria rápida que guarda datos usados recientemente.

$$\text{localidad temporal} + \text{localidad espacial} \quad \Rightarrow \quad \text{mejor rendimiento.}$$

Este caso es hermoso: una palabra francesa entra al inglés técnico y regresa a muchas lenguas como anglicismo computacional.

No todo préstamo viaja en línea recta.

Queue, *stack*, *file*: la estructura y la palabra

Orden temporal

- *stack*: pila, regla LIFO; el último en entrar es el primero en salir.
- *queue*: cola, regla FIFO; el primero en entrar es el primero en salir.

Orden estructural

- *file*: fila/cola; conserva la idea francesa de espera ordenada.
- *heap*: montículo; acceso por prioridad o estructura de memoria.

Nombrar una estructura de datos no es decorativo: fija una disciplina de acceso, una complejidad esperada y una lógica de ejecución.

Algoritmo con cola: BFS

```
Algorithm BFS(G,s)
Input: graph G, source vertex s
Output: distance from s to each reachable vertex
for each vertex v do dist[v] <- infinity
dist[s] <- 0
Q <- empty queue
enqueue(Q,s)
while Q is not empty do
    u <- dequeue(Q)
    for each neighbor v of u do
        if dist[v] == infinity then
            dist[v] <- dist[u] + 1
            enqueue(Q,v)
return dist
```

BFS: lógica de descubrimiento

El invariante de BFS es semántico: cuando un vértice sale de la cola, su distancia ya es mínima.

$$\text{dist}(v) = \text{mín}\{\text{longitud de camino de } s \text{ a } v\}.$$

El término *queue* no es accesorio. La cola impone el orden temporal que hace verdadera la prueba.

- La estructura de datos determina la lógica del algoritmo.
- La lógica justifica la complejidad $O(|V| + |E|)$.

Traducción, préstamo y precisión

Una regla práctica para el aula y la escritura técnica:

- traducir cuando el término español es estable: muestra, varianza, sesgo, función de pérdida;
- conservar cuando el ecosistema técnico lo exige: *pipeline*, *benchmark*, *overfitting*, *backpropagation*;
- explicar siempre la operación matemática detrás de la palabra;
- evitar anglicismos vacíos: *random*, *smart*, *robust* deben volverse hipótesis, métricas o definiciones.

La elegancia técnica no consiste en hablar en inglés: consiste en saber qué operación estamos nombrando.

El Teorema del Estadístico Inconsciente

Una broma pedagógica: calcular como si uno no supiera que $g(X)$ tiene su propia ley.

En inglés se llama *Law of the Unconscious Statistician* (LOTUS). La broma es fina: el "estadístico inconsciente" no se detiene a construir primero la distribución de la variable transformada

$$Y = g(X).$$

Trabaja directamente con la ley de X.

No es ignorancia: es una forma compacta de cambio de variable probabilístico. La ley de X ya contiene toda la información necesaria para promediar cualquier función medible de X.

La conciencia que se evita: transformar la ley

Si X es una variable aleatoria y queremos $\mathbb{E}[g(X)]$, el camino largo sería:

$$X \quad \longmapsto \quad Y = g(X) \quad \longmapsto \quad \mu_Y \quad \longmapsto \quad \int y \, \mu_Y(dy).$$

LOTUS permite usar el camino corto:

$$X \quad \longmapsto \quad \mu_X \quad \longmapsto \quad \int g(x) \, \mu_X(dx).$$

La frase “inconsciente” viene de ahí: se calcula la esperanza de la variable transformada sin escribir explícitamente la función de repartición, la masa o la densidad de $g(X)$.

El objeto correcto no es siempre una densidad; es la ley.

Tres versiones de una misma identidad

En el caso discreto, si X toma valores x_i,

$$\mathbb{E}[g(X)] = \sum_i g(x_i)\,\mathbb{P}(X = x_i).$$

Si X tiene densidad f_X respecto de Lebesgue,

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x) f_X(x)\,dx.$$

La forma limpia, que incluye ambos casos, es la de medida:

$$\mathbb{E}[g(X)] = \int_{\mathbb{R}} g(x)\,\mathbb{P}_X(dx),$$

donde $\mathbb{P}_X$ es la ley de X. En español conviene decir: integrar respecto de la ley o distribución de X, no necesariamente respecto de una densidad.

Ejemplo: X^2 sin hallar la ley de X^2

Sea $X \sim \mathrm{Unif}(0,1)$. Queremos calcular

$$\mathbb{E}[X^2].$$

El camino largo encuentra primero la ley de $Y = X^2$. El camino LOTUS simplemente hace

$$\mathbb{E}[X^2] = \int_0^1 x^2 \, dx = \frac{1}{3}.$$

En pseudocódigo conceptual:

```
Input: law mu_X, measurable function g
Output: expected value of g(X)
return Integral of g(x) with respect to mu_X(dx)
```

La notación evita una mala traducción: no se habla de “densidad” salvo que exista una densidad. Para variables discretas, se usa masa; para el caso general, ley.

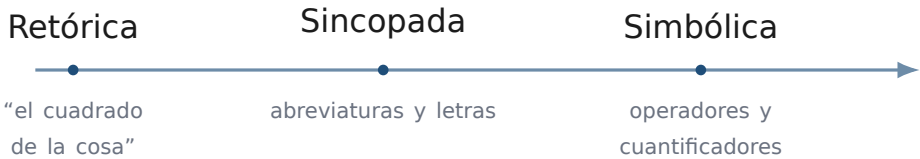
Simbología matemática: una historia de compresión

Un símbolo no sustituye una palabra: condensa una operación mental.

Antes de la notación moderna, una demostración se escribía casi como una prosa: “si esto es verdadero, entonces aquello se sigue”. La simbología matemática nació cuando esa prosa comenzó a volverse demasiado lenta para el cálculo, la prueba y la transmisión internacional.

La historia de los símbolos es una historia de compresión: pasar de frases largas a signos capaces de organizar inferencias, algoritmos y objetos abstractos.

Tres edades de la escritura matemática



El tránsito no fue lineal. Durante siglos convivieron palabras, abreviaturas, diagramas, letras y signos. La notación moderna aparece cuando el símbolo deja de ser una decoración y se vuelve gramática: indica qué se supone, qué se cuantifica, qué se deduce y qué se define.

Cajori; Schlimm, “Where Mathematical Symbols Come From”; MAA Convergence.

Por lo tanto: ∴ como puntuación de la prueba



El signo ∴ no es exactamente una conectiva lógica interna. Funciona más bien como puntuación argumentativa: marca que una línea es conclusión de las anteriores.

Históricamente se asocia con Johann Rahn, quien usó el triángulo de puntos en *Teutsche Algebra* (1659). Su éxito fue práctico: tres puntos bastan para decir “por lo tanto” sin romper el flujo algebraico.

$$x > 2 \quad \Rightarrow \quad x^2 > 4,$$

$$x > 2, \quad x \in \mathbb{R} \quad \therefore \quad x^2 > 4.$$

Cajori; MAA, “Math Origins: The Logical Ideas”.

Implicar no es concluir

$P \Rightarrow Q$
conectiva dentro de una proposición

demostración →

$P;\ P \Rightarrow Q;\ \therefore Q$
paso externo de una prueba

$P \Rightarrow Q$ afirma una relación condicional: si P es verdadera, entonces Q lo es. En cambio, $\therefore Q$ señala que, dado lo escrito antes, hemos alcanzado una conclusión.

Por eso conviene enseñar a leer:

$P \Rightarrow Q$ como "P implica Q", $\therefore Q$ como "por lo tanto, Q".

Existe: del verbo al cuantificador

$$\exists x\ P(x)$$

El símbolo ∃ representa la frase “existe al menos un objeto”. Su genealogía moderna pasa por la lógica simbólica del siglo XIX: Frege introdujo la cuantificación como operación formal de ligar variables; Peirce y Mitchell usaron signos tipo Π y Σ; Peano difundió una notación lineal y adoptó ∃ para la existencia.

El punto profundo no es tipográfico. Es lógico: una oración abierta $P(x)$ se convierte en una proposición cerrada cuando se decide el alcance de la variable.

$$P(x) : x^2 = 2, \qquad \exists x \in \mathbb{R}\ x^2 = 2.$$

Stanford Encyclopedia of Philosophy, “Quantifiers and Quantification”; Peano; Frege; Peirce.

Para todo: universalidad como disciplina

$$\forall x\ P(x)$$

∀ se lee “para todo”. Es una A invertida, asociada a *all*. La notación moderna se consolidó mucho después de que la idea lógica estuviera clara: Aristóteles ya distinguía “todos”, “algunos” y “ninguno”, pero la matemática moderna necesitó un símbolo capaz de ligar variables dentro de fórmulas largas.

El signo ∀ no dice simplemente “muchos”. Dice: no hay excepción dentro del dominio declarado.

$$\forall x \in \mathbb{R}, \quad x^2 \geq 0.$$

La fuerza matemática está en el dominio: $\forall x \in \mathbb{R}$ no es lo mismo que $\forall x \in \mathbb{C}$ cuando aparece una desigualdad de orden.

Gentzen introdujo el símbolo ∀ por analogía con ∃; véase literatura histórica sobre cuantificadores.

El orden de ∀ y ∃ cambia el mundo

Una de las primeras lecciones profundas de la simbología es que el orden de los cuantificadores no es decoración.

Continuidad puntual:

$$\forall x \in \mathbb{R}\ \forall \varepsilon > 0\ \exists \delta > 0\ \forall h,\ |h| < \delta \Rightarrow |f(x+h) - f(x)| < \varepsilon.$$

Continuidad uniforme:

$$\forall \varepsilon > 0\ \exists \delta > 0\ \forall x \in \mathbb{R}\ \forall h,\ |h| < \delta \Rightarrow |f(x+h) - f(x)| < \varepsilon.$$

En la primera, δ puede depender de x. En la segunda, δ debe servir para todos los puntos. La diferencia filosófica es enorme: pasar de lo local a lo global.

Probabilidad: cuantificadores sobre eventos y tiempos

La probabilidad moderna usa cuantificadores para distinguir modos de convergencia.

Convergencia en probabilidad:

$$X_n \to X \text{ en probabilidad} \quad \Longleftrightarrow \quad \forall \varepsilon > 0, \mathbb{P}(|X_n - X| > \varepsilon) \to 0.$$

Convergencia casi segura:

$$\mathbb{P}\Big(\{\omega : \forall \varepsilon > 0 \; \exists N(\omega) \; \forall n \geq N(\omega), \; |X_n(\omega) - X(\omega)| < \varepsilon\}\Big) = 1.$$

La segunda afirmación es más fuerte porque el índice de estabilidad N puede depender de la trayectoria ω, pero la propiedad debe cumplirse en un evento de probabilidad uno.

Pertenecer no es estar contenido

$$x \in A \qquad A \subset B$$

Peano introdujo un epsilon para la relación de pertenencia en *Arithmetices principia, nova methodo exposita* (1889). El signo se lee como "x es elemento de A". Históricamente se conectó con *est*: "es".

La precisión importa:

$$2 \in \mathbb{N}, \qquad \{2\} \subset \mathbb{N}, \qquad 2 \notin \mathbb{N}.$$

En computación esta distinción reaparece como diferencia entre objeto y colección: un entero, una lista con un entero, un tipo y una clase no son intercambiables.

MacTutor, "Earliest Uses of Symbols of Set Theory and Logic"; Peano, 1889.

Definir no es demostrar igualdad

$$F(x) := \mathbb{P}(X \leq x)$$

El símbolo := o ≔ separa una definición de una afirmación. Una igualdad puede ser verdadera o falsa; una definición introduce una convención que fija el significado posterior.

Ejemplo probabilístico:

$$F_X(x) := \mu_X((-\infty, x])$$

no se prueba: se adopta como definición de la función de repartición. Después sí se prueban propiedades:

F_X es no decreciente, continua por la derecha y con límites $0, 1$.

En pseudocódigo, la misma separación aparece entre = como asignación y == como comparación.

Negación, contraejemplos y el arte de negar bien

La simbología también enseña a negar con precisión.

$$\neg(\forall x \in A\ P(x)) \quad \Longleftrightarrow \quad \exists x \in A\ \neg P(x).$$

$$\neg(\exists x \in A\ P(x)) \quad \Longleftrightarrow \quad \forall x \in A\ \neg P(x).$$

Por eso un solo contraejemplo destruye un “para todo”, pero no destruye un “existe”. Esta es una de las primeras formas en que la lógica simbólica mejora la escritura matemática: obliga a distinguir universalidad, existencia y falsación.

Ejemplo:

$$\neg(\forall x \in \mathbb{R},\ x^2 > 0) \quad \text{porque} \quad x = 0.$$

Demostrar no es lo mismo que ser verdadero

$$\Gamma \vdash \varphi \qquad\qquad \Gamma \models \varphi$$

La lógica moderna distingue dos planos.

$\Gamma \vdash \varphi$ significa: hay una derivación formal de φ a partir de Γ dentro de un sistema de reglas.

$\Gamma \models \varphi$ significa: en todo modelo donde las premisas Γ son verdaderas, la conclusión φ también lo es.

Esta diferencia entra directamente en computación: un programa puede estar probado dentro de un sistema formal, y además puede tener una semántica operacional, denotacional o probabilística que interpreta qué significa ejecutarlo.

La simbología como pseudocódigo de la demostración

```
Require: premise P, implication P => Q
Ensure: conclusion Q
if P is established then
    apply implication P => Q
    return Q
else
    proof has no right to conclude Q
end if
```

El razonamiento matemático tiene estructura algorítmica: hipótesis, reglas de inferencia, variables ligadas, invariantes, casos y conclusión.

Por eso la buena notación no sólo hace más corto el texto. Hace visible la arquitectura lógica de una prueba y prepara el paso hacia algoritmos, tipos, verificadores y lenguajes formales.

Ejemplo completo: de una frase a una fórmula

Frase informal:

Si una sucesión converge casi seguramente a X, entonces converge en probabilidad a X.

Lectura simbólica:

$$\Big[\mathbb{P}\{\omega : X_n(\omega) \to X(\omega)\} = 1\Big] \quad \Rightarrow \quad \Big[\forall \varepsilon > 0, \mathbb{P}(|X_n - X| > \varepsilon) \to 0\Big].$$

La fórmula no es adorno. Separa tres niveles: trayectorias ω, eventos medibles y límites de probabilidades. Esa separación es la razón por la cual la notación probabilística moderna es tan poderosa.

Cierre de la sección: símbolos como tecnología de pensamiento

La simbología matemática no elimina el lenguaje: lo disciplina.

$\forall$, $\exists$, $\Rightarrow$, $\therefore$, $\in$, $:=$, $\vdash$ y $\models$ son pequeñas máquinas de precisión. Cada una dice qué tipo de acto matemático estamos realizando: definir, suponer, cuantificar, pertenecer, inferir, concluir o demostrar.

La historia de la notación es, por eso, una historia de maduración: la matemática aprende a escribir de tal modo que sus frases puedan convertirse en pruebas, sus pruebas en algoritmos, y sus algoritmos en objetos verificables.

Mini-glosario operativo, sin traducir mecánicamente

Anglicismos técnicos

- *fit*: ajustar; pregunta por el criterio de ajuste.
- *loss*: pérdida o costo; pregunta qué error penalizamos.
- *pipeline*: flujo de transformaciones; pregunta qué se compone y en qué orden.
- *debug*: depurar; pregunta qué invariante dejó de cumplirse.

Francesismos y cultura matemática

- *ensemble*: conjunto o colectivo; pregunta qué une a los objetos.
- *logiciel*: software; pregunta por la función del sistema.
- *benchmark*: prueba comparativa; pregunta contra qué se mide.
- *cadlag*: derecha continua, límites por la izquierda; pregunta por la forma de las trayectorias.

Conclusión: del préstamo al concepto

Los anglicismos y francesismos útiles sobreviven porque hacen trabajo intelectual.

- El francés dejó una huella fuerte en la matemática estructural y en la informática como disciplina de la información.
- El inglés organizó la cultura contemporánea del software, los datos, ML, estándares y documentación.
- El pseudocódigo mezcla lenguas para volver visible la lógica de un procedimiento.
- C muestra cómo una lengua mínima puede transformar conceptos en máquina.

La pregunta crítica no es “¿de qué lengua viene?”, sino: ¿qué operación permite pensar con mayor precisión?

Cierre

Una palabra técnica es buena cuando reduce ambigüedad y aumenta demostrabilidad.

En matemáticas aplicadas y computación, el vocabulario ideal no busca sonar sofisticado. Busca permitir que una idea pase de la intuición al algoritmo, del algoritmo al código, y del código a una afirmación verificable.

Referencias breves I

- Dennis M. Ritchie, *The Development of the C Language*, Bell Labs, 1993.
- B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice Hall, 1988.
- J. W. Backus et al., *Report on the Algorithmic Language ALGOL 60*, CACM, 1960.
- D. E. Knuth, *The Art of Computer Programming*, Addison-Wesley.
- C. A. R. Hoare, *An Axiomatic Basis for Computer Programming*, CACM, 1969.
- Académie française: *logiciel*; recomendaciones sobre anglicismos informáticos.
- Jacques Perret: propuesta de *ordinateur* para IBM France, 1955.
- Philippe Dreyfus: uso moderno de *informatique*, 1962.

Referencias breves II

- Jacob Bernoulli, *Ars conjectandi*, Basel, 1713.
- MAA Convergence: historical notes on Bernoulli's *Ars Conjectandi*.
- Wikimedia Commons: *Ars Conjectandi of Jakob Bernoulli, 1713*, public-domain reproduction.
- P.-S. Laplace, *Théorie analytique des probabilités*, 1812.
- S.-D. Poisson, *Recherches sur la probabilité des jugements*, 1837.
- L. Bachelier, *Théorie de la spéculation*, Annales scientifiques de l'École Normale Supérieure, 1900.
- É. Borel, H. Lebesgue y P. Lévy: medida, integración y procesos estocásticos en la tradición francesa.
- *Fundamenta Mathematicae* y *Studia Mathematica*: política editorial multilingüe de la escuela polaca.
- K. Kuratowski, *Mathematics in Poland Today*, Canadian Mathematical Bulletin, 1965.
- Florian Cajori, *A History of Mathematical Notations*, Open Court, 1928–1929.
- MAA Convergence, *Math Origins: The Logical Symbols* and *The Logical Ideas*.
- Stanford Encyclopedia of Philosophy, *Quantifiers and Quantification*; *The Notation in Principia Mathematica*.
- MacTutor History of Mathematics, *Earliest Uses of Symbols of Set Theory and Logic*.
- D. Schlimm, *Where Mathematical Symbols Come From*, 2025.